Universidad Nacional Autónoma de México
Escuela Nacional de Estudios Superiores, Unidad Juriquilla

Computadoras y Programación

Tarea 7 / Ejercicio 7

Subdiagramas y diagramas estructurados

Alumno: Daniel Rodríguez Correa
Fecha: 29 de octubre de 2026

Ejercicio 1. Paquetería: intentos de entrega

Herramienta: diagrama de flujo

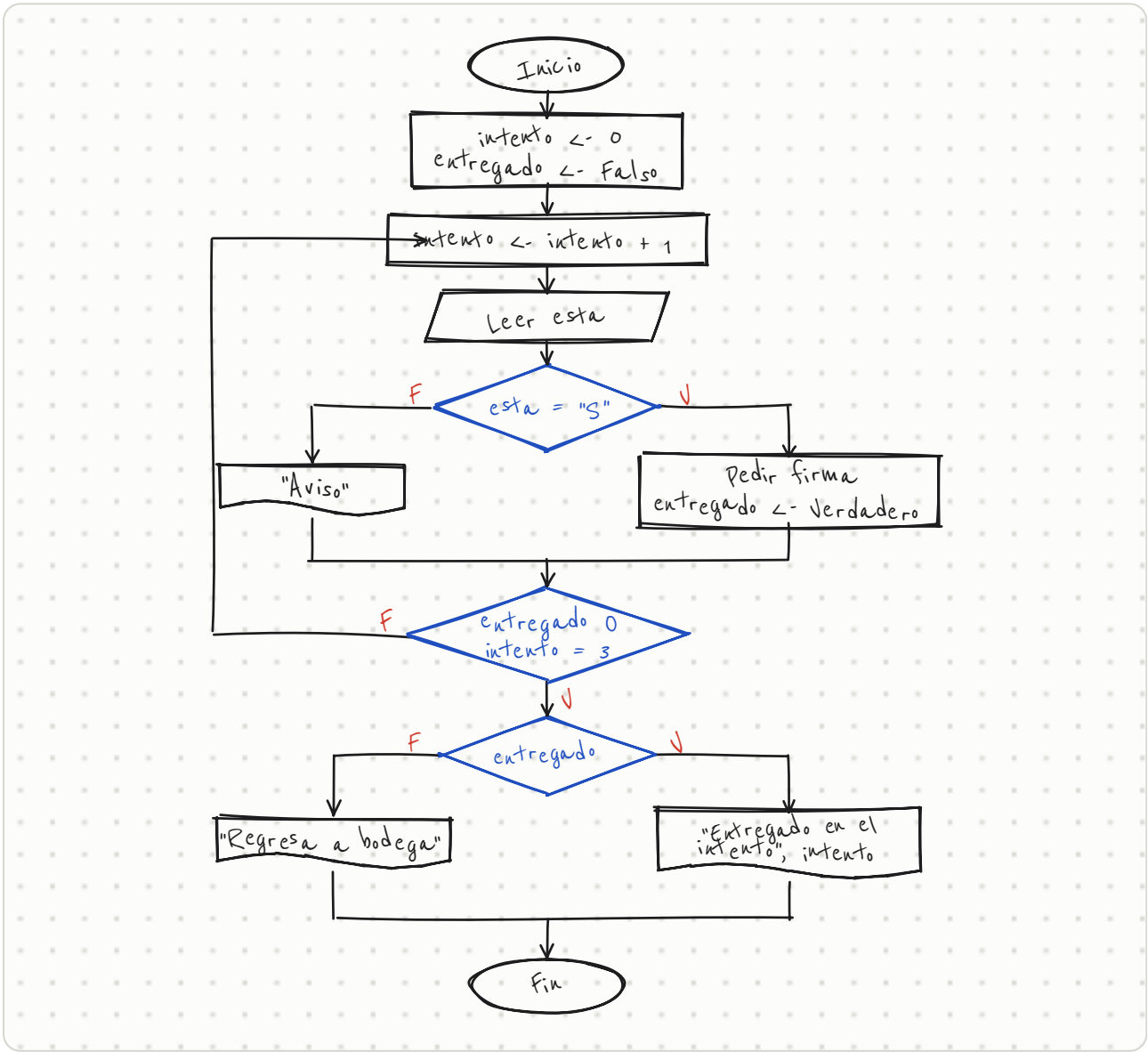
Análisis

En este problema lo único que el programa necesita saber, en cada intento, es si el cliente está o no (lo contesto con S o N). Como el repartidor puede intentar varias veces, usé un ciclo Repetir: en cada vuelta le sumo 1 al contador de intentos y pregunto si el cliente está. Si está, se pide la firma y marco el paquete como entregado; si no está, se deja un aviso.

El ciclo se termina por dos razones: porque ya se entregó o porque ya se hicieron los 3 intentos. Por eso, al salir del ciclo vuelvo a revisar si se entregó, para mostrar "Entregado en el intento n" o "Regresa a bodega".

Escogí el diagrama de flujo porque la idea es capacitar a un repartidor nuevo, y con las flechas se ve muy claro el orden de los pasos y cómo se regresa a intentar otra vez. Alguien que no sabe programar lo puede seguir sin problema.

Diagrama de flujo



Programa en PSeInt

```
Proceso Paqueteria
    Definir intento Como Entero;
    Definir esta Como Caracter;
    Definir entregado Como Logico;
    intento ← 0;
    entregado ← Falso;
    Repetir
        intento ← intento + 1;
        Escribir "Intento ", intento, ". ¿Está el cliente? (S/N):";
        Leer esta;
        Si esta = "S" Entonces
            Escribir "Firma recibida";
            entregado ← Verdadero;
        SiNo
            Escribir "Aviso";
        FinSi
    Hasta Que entregado O intento = 3
    Si entregado Entonces
        Escribir "Entregado en el intento ", intento;
    SiNo
        Escribir "Regresa a bodega";
    FinSi
FinProceso
```

Captura del programa en PSeInt.

Ejecución en PSeInt

```
*** Ejecución Iniciada. ***
Intento 1. ¿Está el cliente? (S/N):
> N
Aviso
Intento 2. ¿Está el cliente? (S/N):
> S
Firma recibida
Entregado en el intento 2
*** Ejecución Finalizada. ***
```

Caso 1

```
*** Ejecución Iniciada. ***
Intento 1. ¿Está el cliente? (S/N):
> S
Firma recibida
Entregado en el intento 1
*** Ejecución Finalizada. ***
```

Caso 2

```
*** Ejecución Iniciada. ***
Intento 1. ¿Está el cliente? (S/N):
> N
Aviso
Intento 2. ¿Está el cliente? (S/N):
> N
Aviso
Intento 3. ¿Está el cliente? (S/N):
> N
Aviso
Regresa a bodega
*** Ejecución Finalizada. ***
```

Caso 3

Ejercicio 2. Videojuego: adivina el número

Herramienta: diagrama de flujo

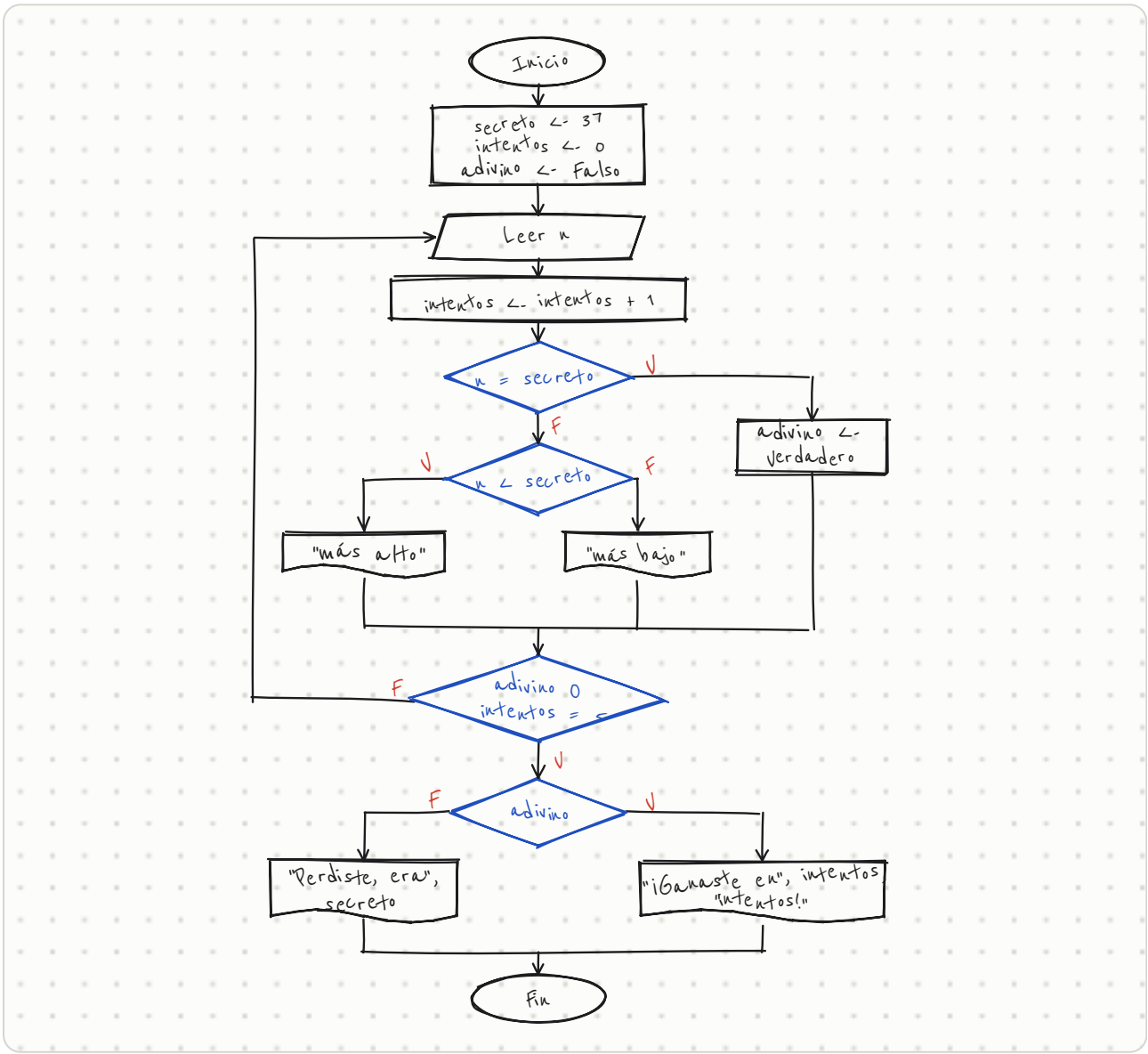
Análisis

El número secreto es 37 y el jugador tiene 5 intentos. Lo único que se lee es el número que escribe en cada intento. Dentro de un Repetir leo el número, sumo un intento y lo comparo con el secreto. Aquí hay tres caminos: si es igual ya adivinó, si es menor el juego le dice "más alto" y si es mayor le dice "más bajo".

El ciclo se repite hasta que adivine o hasta que llegue a 5 intentos. Después muestro si ganó (y en cuántos intentos) o cuál era el número.

Usé diagrama de flujo porque el juego es una serie de turnos que se repite, y en el dibujo se ve bien el regreso al siguiente intento, la comparación de tres caminos y las dos maneras de terminar.

Diagrama de flujo



Programa en PSeInt

```
Proceso AdivinaNumero
    Definir secreto, n, intentos Como Entero;
    Definir adivino Como Logico;
    secreto ← 37;
    intentos ← 0;
    adivino ← Falso;
    Repetir
        Escribir "Intento ", intentos + 1, " de 5. Tu número:";
        Leer n;
        intentos ← intentos + 1;
        Si n = secreto Entonces
            adivino ← Verdadero;
        SiNo
            Si n < secreto Entonces
                Escribir "más alto";
            SiNo
                Escribir "más bajo";
            FinSi
        FinSi
    Hasta Que adivino O intentos = 5
    Si adivino Entonces
        Escribir "¡Ganaste en ", intentos, " intentos!";
    SiNo
        Escribir "Perdiste, era ", secreto;
    FinSi
FinProceso
```

Captura del programa en PSeInt.

Ejecución en PSeInt

```
*** Ejecución Iniciada. ***
Intento 1 de 5. Tu número:
> 50
más bajo
Intento 2 de 5. Tu número:
> 25
más alto
Intento 3 de 5. Tu número:
> 37
¡Ganaste en 3 intentos!
*** Ejecución Finalizada. ***
```

Caso 1

```
*** Ejecución Iniciada. ***
Intento 1 de 5. Tu número:
> 10
más alto
Intento 2 de 5. Tu número:
> 20
más alto
Intento 3 de 5. Tu número:
> 30
más alto
Intento 4 de 5. Tu número:
> 40
más bajo
Intento 5 de 5. Tu número:
> 45
más bajo
Perdiste, era 37
*** Ejecución Finalizada. ***
```

Caso 2

Ejercicio 3. Mundial: goles de un grupo

Herramienta: diagrama Nassi-Shneiderman

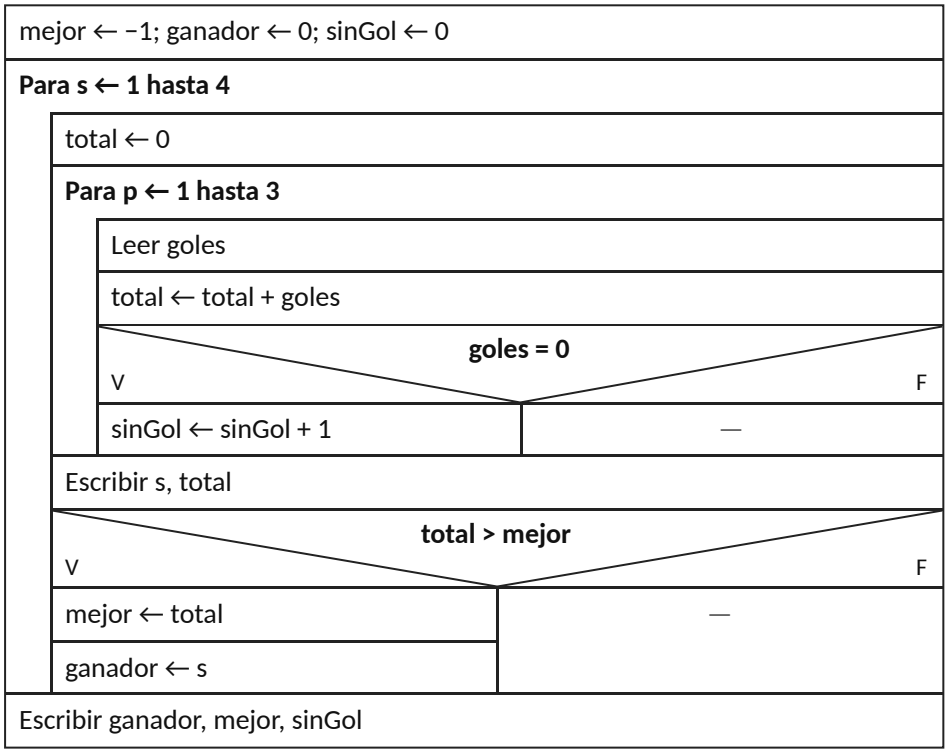
Análisis

Hay 4 selecciones y cada una juega 3 partidos, así que se leen 12 números (los goles). Lo resolví con dos ciclos Para, uno dentro del otro: el de afuera recorre las selecciones y el de adentro sus 3 partidos.

Lo más importante fue decidir dónde reiniciar cada contador. El total de goles se pone en 0 al empezar cada selección, pero el contador de partidos sin gol no se reinicia porque es de todo el grupo. Al terminar los 3 partidos de una selección muestro su total y lo comparo con el mejor que llevo; mejor empieza en −1 para que cualquier total, aunque sea 0, pueda quedar como el mayor. Al final muestro qué selección metió más goles y cuántos partidos terminaron sin gol.

Elegí Nassi-Shneiderman porque tiene ciclos anidados, y con las cajas se ve exactamente en qué nivel va cada instrucción, que es justo lo que había que cuidar.

Diagrama Nassi-Shneiderman



Programa en PSeInt

```
Proceso GolesGrupo
    Definir s, p, goles, total, mejor, ganador, sinGol Como Entero;
    mejor ← -1;
    ganador ← 0;
    sinGol ← 0;
    Para s ← 1 Hasta 4 Con Paso 1 Hacer
        total ← 0;
        Para p ← 1 Hasta 3 Con Paso 1 Hacer
            Escribir "Selección ", s, ", partido ", p, ". Goles:";
            Leer goles;
            total ← total + goles;
            Si goles = 0 Entonces
                sinGol ← sinGol + 1;
            FinSi
        FinPara
        Escribir "Selección ", s, ": ", total, " goles";
        Si total > mejor Entonces
            mejor ← total;
            ganador ← s;
        FinSi
    FinPara
    Escribir "Más goles: selección ", ganador, " (", mejor, ")";
    Escribir "Partidos sin gol: ", sinGol;
FinProceso
```

Captura del programa en PSeInt.

Ejecución en PSeInt

```
*** Ejecución Iniciada. ***
Selección 1, partido 1. Goles:
> 2
Selección 1, partido 2. Goles:
> 1
Selección 1, partido 3. Goles:
> 0
Selección 1: 3 goles
Selección 2, partido 1. Goles:
> 0
Selección 2, partido 2. Goles:
> 0
Selección 2, partido 3. Goles:
> 1
Selección 2: 1 goles
Selección 3, partido 1. Goles:
> 3
Selección 3, partido 2. Goles:
> 2
Selección 3, partido 3. Goles:
> 2
Selección 3: 7 goles
Selección 4, partido 1. Goles:
> 1
Selección 4, partido 2. Goles:
> 0
Selección 4, partido 3. Goles:
> 0
Selección 4: 1 goles
Más goles: selección 3 (7)
Partidos sin gol: 5
*** Ejecución Finalizada. ***
```

Caso 1

Ejercicio 4. Supermercado: menú de la caja

Herramienta: diagrama Nassi-Shneiderman (Repetir con Segun)

Análisis

La caja muestra un menú y el usuario elige una opción; si elige la 1 también se lee el precio. Todo va dentro de un Repetir que termina cuando la opción es 4, y adentro hay un Segun con un caso para cada opción.

Para poder cancelar solo el último producto guardé su precio en la variable ultimo. Cuando se cancela, resto ese precio del total, quito un artículo y pongo ultimo en 0, así ya no se puede volver a cancelar. Si ultimo vale 0 el programa dice "Nada que cancelar". La opción 3 muestra el total y la cantidad de artículos, y la 4 muestra lo que hay que pagar y sale.

Usé Nassi-Shneiderman porque hay un Segun dentro de un Repetir y además un Si dentro de uno de los casos. En N-S cada estructura queda como una caja dentro de otra y se pasa casi directo a PSeInt.

Diagrama Nassi-Shneiderman

<table>
<tr><td colspan="6">total ← 0; arts ← 0; ultimo ← 0</td></tr>
<tr><td colspan="6">Escribir menú</td></tr>
<tr><td colspan="6">Leer op</td></tr>
<tr><td colspan="6">Segun op</td></tr>
<tr><td>1</td><td colspan="2">2</td><td>3</td><td>4</td><td>De otro modo</td></tr>
<tr><td>Leer precio</td><td colspan="2" rowspan="5">ultimo > 0
V F</td><td rowspan="5">Escribir "Total: $", total, arts</td><td rowspan="5">Escribir "Pagar $", total, arts</td><td rowspan="5">Escribir "Opción no válida"</td></tr>
<tr><td>total ← total + precio</td><td>total ← total − ultimo</td><td rowspan="4">Escribir "Nada que cancelar"</td></tr>
<tr><td>arts ← arts + 1</td><td>arts ← arts − 1</td></tr>
<tr><td>ultimo ← precio</td><td>Escribir "Producto cancelado", ultimo</td></tr>
<tr><td></td><td>ultimo ← 0</td></tr>
<tr><td colspan="6">Hasta que op = 4</td></tr>
</table>

Programa en PSeInt

```
Proceso MenuCaja
    Definir op, arts Como Entero;
    Definir precio, total, ultimo Como Real;
    total ← 0;
    arts ← 0;
    ultimo ← 0;
    Repetir
        Escribir "1 Agregar  2 Cancelar último  3 Ver total  4 Pagar y salir";
        Leer op;
        Segun op Hacer
            1:
                Escribir "Precio:";
                Leer precio;
                total ← total + precio;
                arts ← arts + 1;
                ultimo ← precio;
            2:
                Si ultimo > 0 Entonces
                    total ← total - ultimo;
                    arts ← arts - 1;
                    Escribir "Producto cancelado (el de $", ultimo, ")";
                    ultimo ← 0;
                SiNo
                    Escribir "Nada que cancelar";
                FinSi
            3:
                Escribir "Total: $", total, ", artículos: ", arts;
            4:
                Escribir "Pagar $", total, ", artículos: ", arts;
            De Otro Modo:
                Escribir "Opción no válida";
        FinSegun
    Hasta Que op = 4
FinProceso
```

Captura del programa en PSeInt.

Ejecución en PSeInt

```
*** Ejecución Iniciada. ***
1 Agregar  2 Cancelar último  3 Ver total  4 Pagar y salir
> 1
Precio:
> 50
1 Agregar  2 Cancelar último  3 Ver total  4 Pagar y salir
> 1
Precio:
> 30
1 Agregar  2 Cancelar último  3 Ver total  4 Pagar y salir
> 2
Producto cancelado (el de $30)
1 Agregar  2 Cancelar último  3 Ver total  4 Pagar y salir
> 2
Nada que cancelar
1 Agregar  2 Cancelar último  3 Ver total  4 Pagar y salir
> 3
Total: $50, artículos: 1
1 Agregar  2 Cancelar último  3 Ver total  4 Pagar y salir
> 4
Pagar $50, artículos: 1
*** Ejecución Finalizada. ***
```

Caso 1

Ejercicio 5. Laboratorio: la computadora no enciende

Herramienta: árbol de decisión

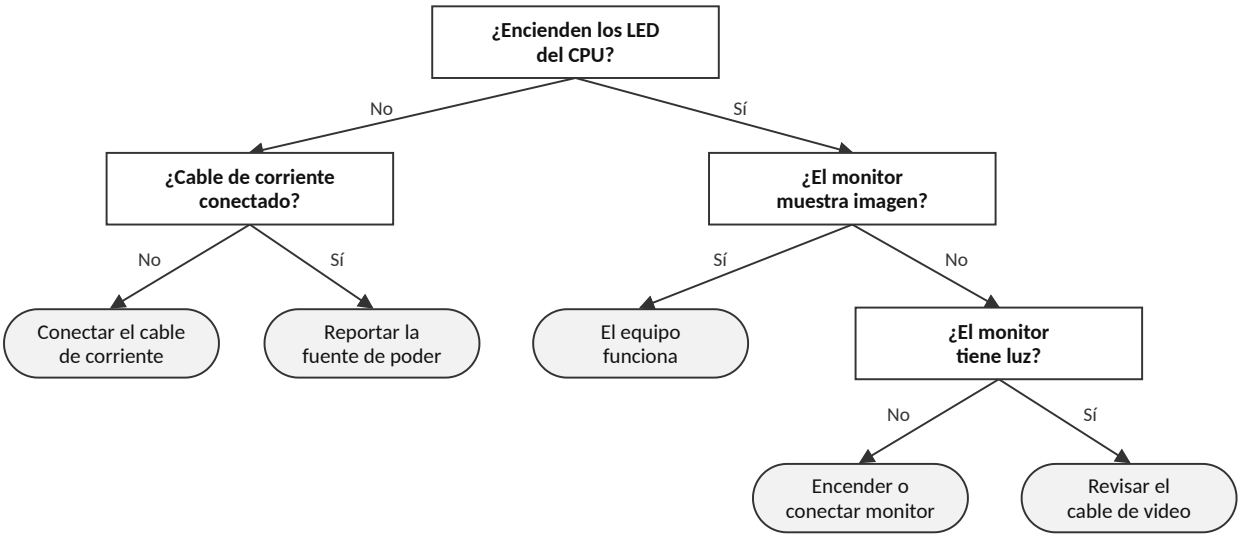
Análisis

Aquí el programa le va haciendo preguntas de Sí o No al personal del laboratorio: si encienden los LED del CPU, si el cable de corriente está conectado, si el monitor muestra imagen y si el monitor tiene luz. Cada pregunta depende de lo que se contestó antes, así que la primera (la raíz) es lo primero que uno revisa en la realidad: los LED.

Antes de dibujar conté las acciones posibles. Son 5, entonces el árbol tiene 5 hojas y 4 preguntas. El programa solo hace las preguntas necesarias para llegar a una indicación.

Escogí el árbol de decisión porque es un diagnóstico: importa el orden de las preguntas y se evitan preguntas que no sirven. Por ejemplo, si los LED no encienden ya no tiene caso revisar el monitor.

Árbol de decisión



Programa en PSeInt

```
Proceso DiagnosticoPC
    Definir led, cable, imagen, luz Como Caracter;
    Escribir "¿Encienden los LED del CPU? (S/N):";
    Leer led;
    Si led = "N" Entonces
        Escribir "¿El cable de corriente está conectado? (S/N):";
        Leer cable;
        Si cable = "N" Entonces
            Escribir "Conectar el cable de corriente";
        SiNo
            Escribir "Reportar la fuente de poder";
        FinSi
    SiNo
        Escribir "¿El monitor muestra imagen? (S/N):";
        Leer imagen;
        Si imagen = "S" Entonces
            Escribir "El equipo funciona";
        SiNo
            Escribir "¿El monitor tiene luz? (S/N):";
            Leer luz;
            Si luz = "N" Entonces
                Escribir "Encender o conectar el monitor";
            SiNo
                Escribir "Revisar el cable de video";
            FinSi
        FinSi
    FinSi
FinProceso
```

Captura del programa en PSeInt.

Ejecución en PSeInt

```
*** Ejecución Iniciada. ***
¿Encienden los LED del CPU? (S/N):
> N
¿El cable de corriente está conectado? (S/N):
> N
Conectar el cable de corriente
*** Ejecución Finalizada. ***
```

Caso 1

```
*** Ejecución Iniciada. ***
¿Encienden los LED del CPU? (S/N):
> N
¿El cable de corriente está conectado? (S/N):
> S
Reportar la fuente de poder
*** Ejecución Finalizada. ***
```

Caso 2

```
*** Ejecución Iniciada. ***
¿Encienden los LED del CPU? (S/N):
> S
¿El monitor muestra imagen? (S/N):
> S
El equipo funciona
*** Ejecución Finalizada. ***
```

Caso 3

```
*** Ejecución Iniciada. ***
¿Encienden los LED del CPU? (S/N):
> S
¿El monitor muestra imagen? (S/N):
> N
¿El monitor tiene luz? (S/N):
> N
Encender o conectar el monitor
*** Ejecución Finalizada. ***
```

Caso 4

```
*** Ejecución Iniciada. ***
¿Encienden los LED del CPU? (S/N):
> S
¿El monitor muestra imagen? (S/N):
> N
¿El monitor tiene luz? (S/N):
> S
Revisar el cable de video
*** Ejecución Finalizada. ***
```

Caso 5

Ejercicio 6. Rutas: ¿cómo llego?

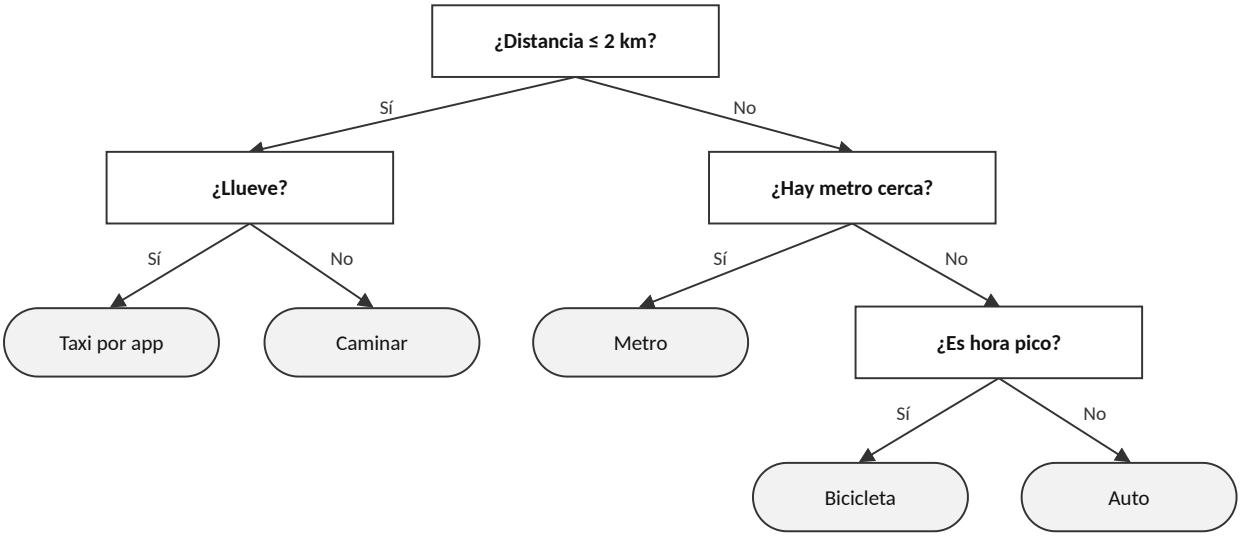
Herramienta: árbol de decisión

Análisis

La app necesita la distancia y, dependiendo de ella, otras respuestas: si llueve, si hay metro cerca o si es hora pico. Puse la distancia como raíz porque de eso depende lo demás: si son 2 km o menos solo importa la lluvia (taxi o caminar); si son más de 2 km se pregunta por el metro y, si no hay, por la hora pico (bicicleta o auto). Son 5 recomendaciones posibles, o sea 5 hojas.

Usé árbol de decisión porque cada pregunta depende de la anterior, y en el árbol se ve que nunca se pregunta por la lluvia en un viaje largo ni por el metro en uno corto.

Árbol de decisión



Programa en PSeInt

```
Proceso RecomendarRuta
    Definir dist Como Real;
    Definir llueve, metro, pico Como Caracter;
    Escribir "Distancia al destino (km):";
    Leer dist;
    Si dist ≤ 2 Entonces
        Escribir "¿Llueve? (S/N):";
        Leer llueve;
        Si llueve = "S" Entonces
            Escribir "Taxi por app";
        SiNo
            Escribir "Caminar";
        FinSi
    SiNo
        Escribir "¿Hay metro cerca? (S/N):";
        Leer metro;
        Si metro = "S" Entonces
            Escribir "Metro";
        SiNo
            Escribir "¿Es hora pico? (S/N):";
            Leer pico;
            Si pico = "S" Entonces
                Escribir "Bicicleta";
            SiNo
                Escribir "Auto";
            FinSi
        FinSi
    FinSi
FinProceso
```

Captura del programa en PSeInt.

Ejecución en PSeInt

```
*** Ejecución Iniciada. ***
Distancia al destino (km):
> 1.5
¿Llueve? (S/N):
> S
Taxi por app
*** Ejecución Finalizada. ***
```

Caso 1

```
*** Ejecución Iniciada. ***
Distancia al destino (km):
> 1.5
¿Llueve? (S/N):
> N
Caminar
*** Ejecución Finalizada. ***
```

Caso 2

```
*** Ejecución Iniciada. ***
Distancia al destino (km):
> 8
¿Hay metro cerca? (S/N):
> S
Metro
*** Ejecución Finalizada. ***
```

Caso 3

```
*** Ejecución Iniciada. ***
Distancia al destino (km):
> 8
¿Hay metro cerca? (S/N):
> N
¿Es hora pico? (S/N):
> S
Bicicleta
*** Ejecución Finalizada. ***
```

Caso 4

```
*** Ejecución Iniciada. ***
Distancia al destino (km):
> 8
¿Hay metro cerca? (S/N):
> N
¿Es hora pico? (S/N):
> N
Auto
*** Ejecución Finalizada. ***
```

Caso 5

Ejercicio 7. Rutas: tarifa de una app de transporte

Herramienta: tabla de decisión

Análisis

Para calcular la tarifa se necesitan la tarifa base, los kilómetros y saber si es hora pico y si llueve. Las condiciones son tres (hora pico, lluvia y viaje de más de 10 km), entonces la tabla completa tiene $2^3 = 8$ reglas.

Si hay hora pico y lluvia, la tarifa se multiplica por 1.5; si solo pasa una de las dos, se multiplica por 1.2, y si no pasa ninguna se cobra normal. Aparte, si el viaje es de más de 10 km se suman $30. En el programa el multiplicador lo hice con Si anidados porque solo se puede aplicar uno, y el cargo con un Si separado porque se suma en cualquier caso.

Elegí la tabla de decisión porque es una regla de cobro y no puede quedar ningún caso sin tarifa; con la tabla reviso las 8 combinaciones y compruebo que ninguna aplica dos multiplicadores al mismo tiempo.

Tabla de decisión completa (8 reglas)

Condiciones	**R1**	**R2**	**R3**	**R4**	**R5**	**R6**	**R7**	**R8**
C1 ¿Hora pico?	S	S	S	S	N	N	N	N
C2 ¿Llueve?	S	S	N	N	S	S	N	N
C3 ¿Viaje > 10 km?	S	N	S	N	S	N	S	N
Acciones								
A1 Tarifa × 1.5	X	X						
A2 Tarifa × 1.2			X	X	X	X		
A3 Tarifa normal							X	X
A4 Cargo de $30	X		X		X		X	

¿Se puede simplificar?

No. Para juntar dos reglas tienen que tener las mismas acciones y ser diferentes en una sola condición. Las parejas que solo cambian en C3 (R1 y R2, R3 y R4, R5 y R6, R7 y R8) no se pueden juntar porque cambia el cargo de $30, y las que solo cambian en C1 o en C2 tienen distinto multiplicador. Las únicas reglas con las mismas acciones son R3 (S N S) y R5 (N S S), pero son diferentes en dos condiciones, así que tampoco se juntan. La tabla se queda con sus 8 reglas.

Programa en PSeInt

```
Proceso TarifaTransporte
    Definir tarifa, km Como Real;
    Definir pico, lluvia Como Caracter;
    Escribir "Tarifa base, km, ¿hora pico? (S/N), ¿llueve? (S/N):";
    Leer tarifa, km, pico, lluvia;
    Si pico = "S" Y lluvia = "S" Entonces
        tarifa ← tarifa * 1.5;
    SiNo
        Si pico = "S" O lluvia = "S" Entonces
            tarifa ← tarifa * 1.2;
        FinSi
    FinSi
    // el cargo va aparte porque se suma en cualquier caso
    Si km > 10 Entonces
        tarifa ← tarifa + 30;
    FinSi
    Escribir "Tarifa: $", tarifa;
FinProceso
```

Captura del programa en PSeInt.

Ejecución en PSeInt

```
*** Ejecución Iniciada. ***
Tarifa base, km, ¿hora pico? (S/N), ¿llueve? (S/N):
> 100
> 12
> S
> S
Tarifa: $180
*** Ejecución Finalizada. ***
```

Caso 1

```
*** Ejecución Iniciada. ***
Tarifa base, km, ¿hora pico? (S/N), ¿llueve? (S/N):
> 100
> 5
> S
> N
Tarifa: $120
*** Ejecución Finalizada. ***
```

Caso 2

```
*** Ejecución Iniciada. ***
Tarifa base, km, ¿hora pico? (S/N), ¿llueve? (S/N):
> 100
> 12
> N
> S
Tarifa: $150
*** Ejecución Finalizada. ***
```

Caso 3

```
*** Ejecución Iniciada. ***
Tarifa base, km, ¿hora pico? (S/N), ¿llueve? (S/N):
> 100
> 5
> N
> N
Tarifa: $100
*** Ejecución Finalizada. ***
```

Caso 4

Ejercicio 8. Videojuego: ¿qué hace el héroe?

Herramienta: tabla de decisión

Análisis

Para saber qué hace el héroe se necesita su vida, si tiene poción y si el enemigo es jefe. Con tres condiciones salen 8 reglas. Si la vida es menor que 30 y tiene poción, la usa; si no tiene poción, huye cuando es jefe y ataca cuando no lo es. Con vida de 30 o más siempre ataca, y si el enemigo es jefe además activa el escudo.

Usé tabla de decisión porque son las reglas de comportamiento del personaje y quería asegurarme de que ningún turno se quedara sin acción; además, con la tabla completa pude ver qué reglas se podían juntar.

Tabla de decisión completa (8 reglas)

Condiciones	**R1**	**R2**	**R3**	**R4**	**R5**	**R6**	**R7**	**R8**
C1 ¿Vida < 30?	S	S	S	S	N	N	N	N
C2 ¿Tiene poción?	S	S	N	N	S	S	N	N
C3 ¿Enemigo jefe?	S	N	S	N	S	N	S	N
Acciones								
A1 Usar poción	X	X						
A2 Huir			X					
A3 Activar escudo					X		X	
A4 Atacar				X	X	X	X	X

Tabla simplificada (5 reglas)

Condiciones	**R1-2**	**R3**	**R4**	**R5-7**	**R6-8**
C1 ¿Vida < 30?	S	S	S	N	N
C2 ¿Tiene poción?	S	N	N	—	—
C3 ¿Enemigo jefe?	—	S	N	S	N
Acciones					
A1 Usar poción	X				
A2 Huir		X			
A3 Activar escudo				X	
A4 Atacar			X	X	X

R1 y R2 tienen la misma acción (usar poción) y solo cambian en C3, así que C3 queda con "—": si tiene poción no importa si es jefe. R5 y R7 hacen lo mismo (escudo y atacar) y solo cambian en C2, igual que R6 y R8 (solo atacar). De 8 reglas quedan 5 y el héroe se comporta igual.

Programa en PSeInt

```
Proceso TurnoHeroe
    Definir vida Como Entero;
    Definir pocion, jefe Como Caracter;
    Escribir "Vida, ¿tiene poción? (S/N), ¿enemigo jefe? (S/N):";
    Leer vida, pocion, jefe;
    Si vida < 30 Entonces
        Si pocion = "S" Entonces
            Escribir "Usar poción";
        SiNo
            Si jefe = "S" Entonces
                Escribir "Huir";
            SiNo
                Escribir "Atacar";
            FinSi
        FinSi
    SiNo
        Si jefe = "S" Entonces
            Escribir "Activar escudo";
        FinSi
        Escribir "Atacar";
    FinSi
FinProceso
```

Captura del programa en PSeInt.

Ejecución en PSeInt

```
*** Ejecución Iniciada. ***
Vida, ¿tiene poción? (S/N), ¿enemigo jefe? (S/N):
> 20
> S
> N
Usar poción
*** Ejecución Finalizada. ***
```

Caso 1

```
*** Ejecución Iniciada. ***
Vida, ¿tiene poción? (S/N), ¿enemigo jefe? (S/N):
> 20
> N
> S
Huir
*** Ejecución Finalizada. ***
```

Caso 2

```
*** Ejecución Iniciada. ***
Vida, ¿tiene poción? (S/N), ¿enemigo jefe? (S/N):
> 20
> N
> N
Atacar
*** Ejecución Finalizada. ***
```

Caso 3

```
*** Ejecución Iniciada. ***
Vida, ¿tiene poción? (S/N), ¿enemigo jefe? (S/N):
> 50
> S
> S
Activar escudo
Atacar
*** Ejecución Finalizada. ***
```

Caso 4

Ejercicios 9 y 10. ¿Qué herramienta usarías?

Para elegir me fijé en qué pregunta tiene que responder cada caso: el diagrama de flujo responde en qué orden pasan las cosas, el N-S cómo se anidan las estructuras, el árbol qué se pregunta primero y la tabla si están cubiertos todos los casos.

Ejercicio 9

Situación	Herramienta	Por qué	Por qué no otra
A. Supermercado: explicar a cajeros nuevos la devolución, paso por paso.	**Diagrama de flujo**	Es un proceso en orden y lo van a leer personas que no programan. Con las flechas se sigue paso por paso.	El N-S es más para programar. El árbol y la tabla solo muestran decisiones, no toda la secuencia.
B. Paquetería: revisar que las comisiones (4 condiciones Sí/No) no dejen casos sin pago.	**Tabla de decisión**	Obliga a escribir las $2^4 = 16$ combinaciones, así se ve si falta un caso o si dos reglas se contradicen.	Con un árbol es fácil saltarse combinaciones. Flujo y N-S muestran el proceso, pero no comprueban que estén todos los casos.
C. Laboratorio: diagnosticar por qué no imprime; cada revisión depende de la anterior.	**Árbol de decisión**	Ordena las revisiones y en cada camino solo se pregunta lo necesario.	La tabla pierde el orden y obliga a poner combinaciones que no aplican.
D. Videojuego: diseñar antes de programar tres ciclos anidados (filas, columnas, capas).	**Nassi-Shneiderman**	Son tres cajas Para, una dentro de otra, sin flechas, y se pasan directo al código.	En diagrama de flujo, tres ciclos anidados llenan el dibujo de flechas de regreso. Árbol y tabla no tienen ciclos.

Ejercicio 10

Situación	Herramienta	Por qué	Por qué no otra
E. Mundial: clasificar por lugar y, solo si hace falta, por diferencia de goles.	**Árbol de decisión**	El lugar va en la raíz y la diferencia de goles solo aparece en las ramas donde se necesita.	La tabla no muestra el orden de las preguntas ni el "solo si hace falta".
F. Casino: documentar el menú del simulador (Repetir, Segun y Si) para pasarlo a C.	**Nassi-Shneiderman**	El Repetir, el Segun y el Si quedan como cajas anidadas que se pasan casi línea por línea a C.	Un diagrama de flujo con un Segun de varios casos dentro de un ciclo se llena de flechas. Árbol y tabla no tienen ciclos.
G. Tienda: comprobar que las promociones del Buen Fin no se contradicen.	**Tabla de decisión**	En cada columna están todas las promociones de una combinación, así que dos descuentos incompatibles en la misma regla se notan luego luego.	En un árbol los casos quedan repartidos en ramas y no se ven dos acciones juntas.
H. Rutas: presentar a los usuarios cómo se calcula una ruta (pedir origen, buscar caminos, elegir el más corto, mostrarlo).	**Diagrama de flujo**	Es una secuencia de pasos para gente que no programa.	El N-S es muy técnico para usuarios. Árbol y tabla muestran decisiones, no un proceso.